

Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное учреждение
высшего образования “Омский государственный технический университет”
Кафедра “Автоматизированные системы обработки информации и
управления”

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
по дисциплине «Кроссплатформенная разработка»
студента Басуева Артёма Дмитриевича группы ПИН-252т

Пояснительная записка
Шифр работы От–2068998–43–ПИН-252т-3 ЛР
Специальность 09.03.04 Программная инженерия

Преподаватель

М. А. Мищенко

Студент

А.Д. Басуев

Омск 2026

РЕФЕРАТ

Отчёт 14 с., 6 рис., 1 табл., 5 источн.

KOTLIN MULTIPLATFORM, COMPOSE MULTIPLATFORM,
NAVIGATION 3, VIEWMODEL, РЕПОЗИТОРИЙ, ЛОКАЛИЗАЦИЯ,
MATERIAL 3, OPENAIRE GRAPH

Объект разработки - кроссплатформенное приложение «ВикиПедия» для чтения статей энциклопедии, разработанное на Kotlin с использованием Compose Multiplatform.

Цель работы - построение каркаса приложения: экрана-списка статей с поиском и постраничной подгрузкой на моковых данных, экрана статьи со ссылками на связанные статьи, экрана избранного, навигации между экранами и локализации интерфейса.

В результате сделано приложение на Compose Multiplatform со списком из 20 статей, поиском по названию, деталями статей и переходом к похожим статьями. Есть светлая и тёмная тема, интерфейс на русском и английском. Приложение собирается для desktop, web и Android.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1. Постановка задачи	5
2. Архитектура приложения	5
3 Модель статьи и репозиторий	6
4. Моковые данные	6
5. Состояние экрана и намерения	7
6. Навигация	7
7. Связанные статьи.....	8
8. Избранное	8
9. Тема	8
10. Локализация.....	8
11. Проверка работы	9
ЗАКЛЮЧЕНИЕ	13
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	14

ВВЕДЕНИЕ

Кроссплатформенная разработка позволяет описать интерфейс и логику приложения один раз и собрать их под несколько платформ. Compose Multiplatform расширяет декларативный подход Jetpack Compose на настольные системы, веб и iOS: общий код лежит в одном модуле, а платформенные точки входа только запускают его.

Предметной областью выбрана энциклопедия: приложение показывает статьи в стиле Википедии. На первой вехе сеть не используется, данные моковые, а доступ к ним скрыт за интерфейсом репозитория, поэтому замена моков сетевым источником не затронет экраны и ViewModel.

Цель работы - разработать каркас приложения: экраны, навигацию, репозиторий, локализацию интерфейса.

Для достижения цели поставлены задачи:

- создать проект Kotlin Multiplatform с общим модулем и настольной точкой входа;
- подготовить моковый набор из двадцати статей;
- реализовать экран-список с поиском и подгрузкой, экран статьи и экран избранного, каждый на связке «ViewModel + Composable»;
- скрыть данные за интерфейсом репозитория;
- организовать навигацию на Navigation 3;
- использовать цвета только из темы, а подписи интерфейса вынести в строковые ресурсы на двух языках;
- добавить автоматическую проверку согласованности локалей.

1. Постановка задачи

Приложение состоит из трёх экранов. Список показывает карточки статей (заголовок, начало текста, две категории), поле поиска по заголовку и тексту и подгружает статьи порциями при прокрутке. Экран статьи содержит заголовок, размер и дату изменения, категории, текст, список связанных статей и ссылку на источник; нажатие на связанную статью открывает её экран поверх текущего. Экран избранного показывает статьи, отмеченные звёздочкой.

Основная сущность предметной области — статья (WikiPage). Её поля приведены в таблице 1.

Таблица 1 — Модель статьи

Поле	Описание
pageid	Уникальный идентификатор статьи
title	Заголовок
extract	Краткое содержание (вводный абзац)
fullurl	Ссылка на оригинал
length	Размер статьи в байтах
touched	Дата и время последнего изменения (ISO 8601)
categories	Категории статьи
links	Ссылки на другие статьи (по заголовку)

Набор полей повторяет данные, которые возвращает программный интерфейс MediaWiki Action API (<https://en.wikipedia.org/w/api.php>) в запросе `action=query` со свойствами `extracts`, `info`, `categories` и `links`. Его планируется подключить на следующих вехах вместо моков.

2. Архитектура приложения

Код разделён по ответственности. Слой данных (data) содержит модель статьи, интерфейс репозитория, моки и их реализацию. Каждый экран (list, detail, favorites) состоит из состояния, намерений, одноразовых событий, ViewModel и Composable-функции. Навигация вынесена в пакет navigation, общие элементы интерфейса - в ui. Поток данных однонаправленный: экран отправляет намерение во ViewModel, ViewModel обращается к репозиторию и обновляет состояние, а экран заново рисуется из нового состояния.

Общий модуль лежит в `shared/src/commonMain` и состоит из

следующих частей:

- App.kt - тема, стек навигации, связка экранов и ViewModel;
- data - модель статьи, моки, интерфейс репозитория, его моковая реализация и хранилище избранного;
- list, detail, favorites - состояние, намерения, события, ViewModel и экран каждого из трёх экранов;
- navigation - маршруты WikiRoute;
- ui - общие элементы (AppScaffold, CategoryChip), фабрика ViewModel и AppContainer;
- composeResources - строковые ресурсы values и values-ru;
- tools/check-strings.py - проверка согласованности локалей.

Состояние хранится во ViewModel в StateFlow, наружу отдаётся поток только для чтения. Действия пользователя приходят в единственный метод onIntent. ViewModel ничего не знает о навигации: переход она сообщает одноразовым событием (Effect), а выполняет его функция App. Репозиторий создаётся в AppContainer, это единственное место, где выбирается его реализация.

3 Модель статьи и репозиторий

Статья описывается классом WikiPage с полями из таблицы 1; категории и ссылки - отдельные классы Category и PageLink с единственным полем title. Доступ к данным закрыт интерфейсом WikiRepository. Он отдаёт поток идентификаторов избранных статей и содержит операции: поиск статей по запросу со смещением и размером порции (результат - порция статей и признак того, что остались ещё), получение статьи по идентификатору, поиск статьи по заголовку (нужен для переходов по ссылкам), получение набора статей по идентификаторам (нужно для избранного) и переключение избранного. Операции получения данных приостанавливаемые: сейчас они отдают моки, а позже будут выполнять сетевой запрос, не блокируя интерфейс. Поиск и подгрузка выражены через смещение и размер порции, как в сетевых интерфейсах.

4. Моковые данные

Моки содержат двадцать статей по информационным технологиям (Kotlin, Android, Linux, Git и другие), у каждой по три категории и по две ссылки на другие статьи набора. Реализация MockWikiRepository хранит этот

список в памяти. При поиске она без учёта регистра выбирает статьи, у которых запрос встречается в заголовке или в тексте, пропускает заданное число элементов, берёт порцию нужного размера и сообщает, остались ли ещё статьи. Для подгрузки следующих порций добавлена имитация сетевой задержки в 400 мс.

5. Состояние экрана и намерения

Экран не обрабатывает нажатия сам: ему передаются состояние и функция обработки намерений. Если передать в экран ViewModel целиком, компилятор Compose признает параметр нестабильным и перестанет пропускать перерисовку. Состояние списка хранит текст запроса, загруженные статьи, идентификаторы избранных, признак идущей подгрузки и признак того, что статьи ещё остались. Намерения пользователя описаны запечатанным интерфейсом: нажатие на статью, изменение запроса, запрос следующей порции и переключение избранного. ViewModel принимает их в единственном методе onIntent: нажатие превращается в одноразовое событие перехода, изменение запроса обновляет состояние и запускает новый поиск, а подгрузка и избранное передаются репозиторию.

Поиск фильтрует статьи по заголовку и тексту без учёта регистра. Новый поиск отменяет идущую подгрузку, чтобы старый результат не добавился к новому списку. Список выводится ленивым столбцом, каждому элементу задан ключ pageId; когда видимыми остаются последние три карточки, экран отправляет намерение LoadMore.

6. Навигация

В Navigation 3 стек маршрутов - обычный список объектов. Маршруты описаны запечатанным интерфейсом WikiRoute: список и избранное - объекты без полей, статья - класс данных с идентификатором, поэтому ошибку в аргументе находит компилятор. В маршрут кладётся только идентификатор, статью экран получает из репозитория. Стек хранится в функции App и начинается со списка: открытие экрана добавляет маршрут, возврат снимает последний.

Хост навигации NavDisplay получает стек маршрутов, обработчик возврата, декораторы записей, спецификации переходов и провайдер записей, который сопоставляет каждому маршруту экран. ViewModel статьи создаётся внутри записи из идентификатора маршрута и репозитория. Явно заданы

анимации прямого и обратного перехода: при возврате экран уезжает в обратную сторону, а не повторяет прямую анимацию.

Декораторы записей привязывают хранилище ViewModel к записи стека: экран статьи получает свой экземпляр, а при возврате назад экземпляр очищается. ViewModel списка создаётся на уровне App и переживает переходы, поэтому поиск и позиция подгрузки сохраняются.

7. Связанные статьи

Связь статьи с другими определяется полем links. Нажатие на ссылку приводит к поиску статьи по заголовку в репозитории; если она найдена, ViewModel отправляет событие, а App добавляет маршрут в стек. Если статьи нет в наборе, ничего не происходит. Так как переход добавляет маршрут, возврат закрывает экраны по одному в обратном порядке.

8. Избранное

Идентификаторы избранных статей хранятся в репозитории как StateFlow, на него подписаны все три ViewModel: звёздочка на карточке, звёздочка на экране статьи и список избранного обновляются одновременно. Избранное хранится в памяти сессии и между запусками не сохраняется.

9. Тема

Цветовая схема выбирается при запуске по системной настройке и переключается кнопкой на экране списка; выбор хранится в состоянии функции App. Схема строится стандартными светлой и тёмной палитрами Material 3 и передаётся в MaterialTheme. Элементы интерфейса читают цвета из MaterialTheme.colorScheme и не задают их числом.

10. Локализация

Подписи интерфейса (15 строк) вынесены в строковые ресурсы: каталог values содержит английские значения, values-ru - русские. По ресурсам создаётся класс Res, поэтому в коде используется сгенерированный идентификатор, и опечатка в имени строки не компилируется. Язык выбирается по локали системы. Строки с параметрами подставляются через stringResource: например, подпись с размером и датой статьи получает число байт и отформатированную дату.

Согласованность локалей проверяет скрипт tools/check-strings.py: он сравнивает наборы ключей и плейсхолдеры (%1\$d, %2\$s) в каждой локали с базовой и находит ссылки Res.string.* на несуществующие ключи.

Локализуется интерфейс, а не данные: заголовки и тексты статей остаются на языке источника.

11. Проверка работы

Приложение собирается для desktop, web (wasmJs и js) и Android.

Команды запуска:

`./gradlew :desktopApp:run` – запуск на десктопе

`./gradlew :webApp:wasmJsBrowserDevelopmentRun` – веб-версия

`./gradlew :androidApp:assembleDebug` – Android-версия

`python tools/check-strings.py` – проверка локализации

По отчёту компилятора Compose все Composable-функции приложения `restartable` и `skippable`, нестабильных параметров нет.

Ниже приведены экраны приложения на desktop.

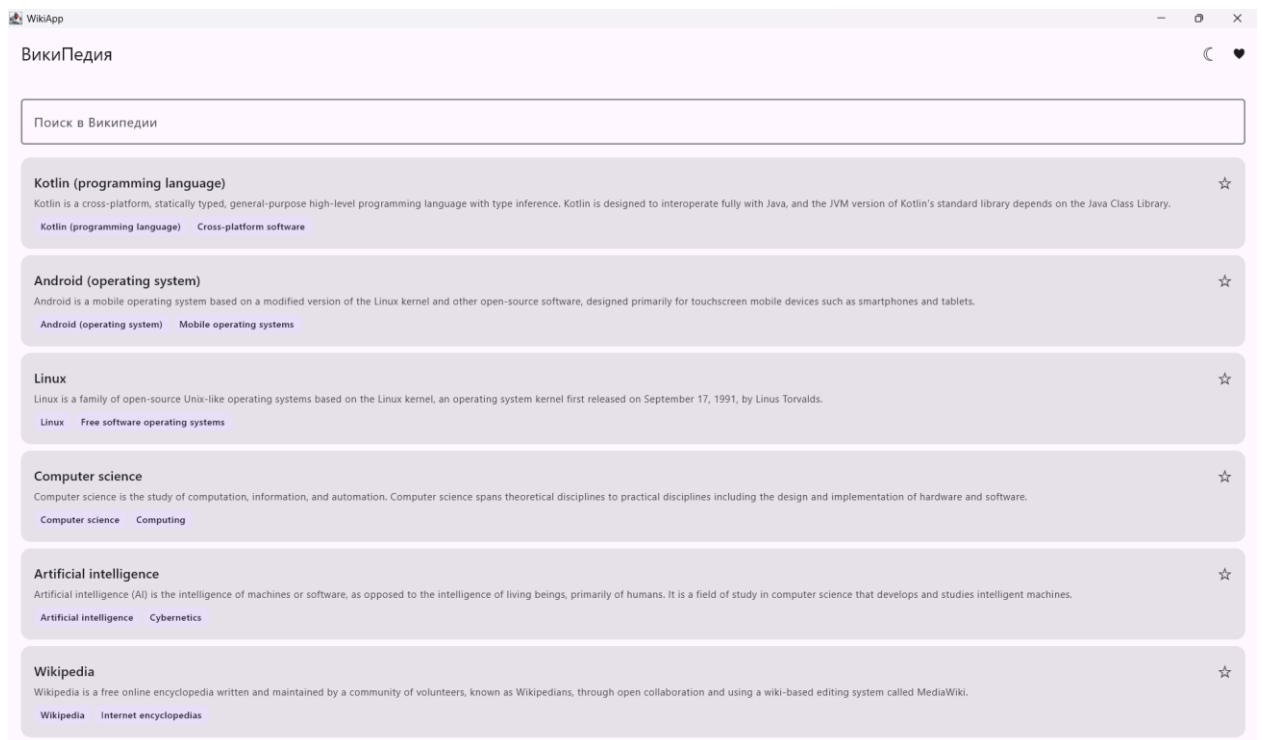


Рисунок 1 – Список статей



Рисунок 2 – Поиск с результатом

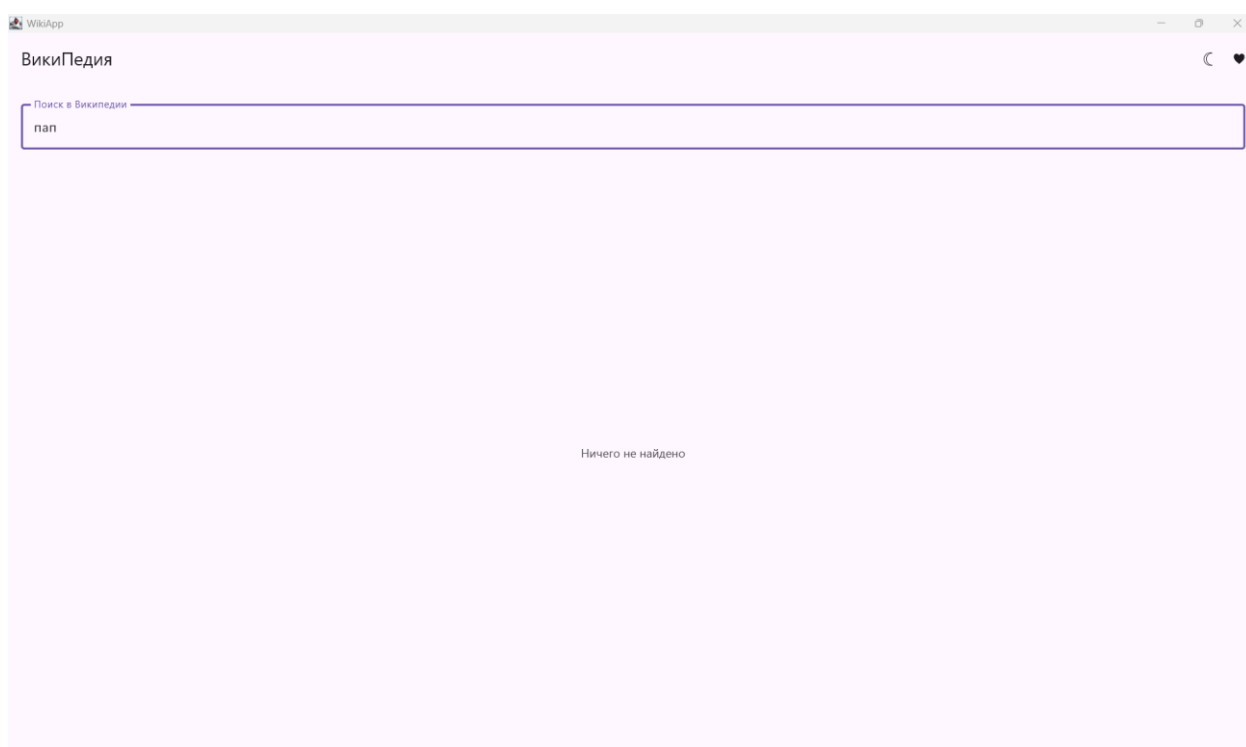


Рисунок 3 – Поиск без результата

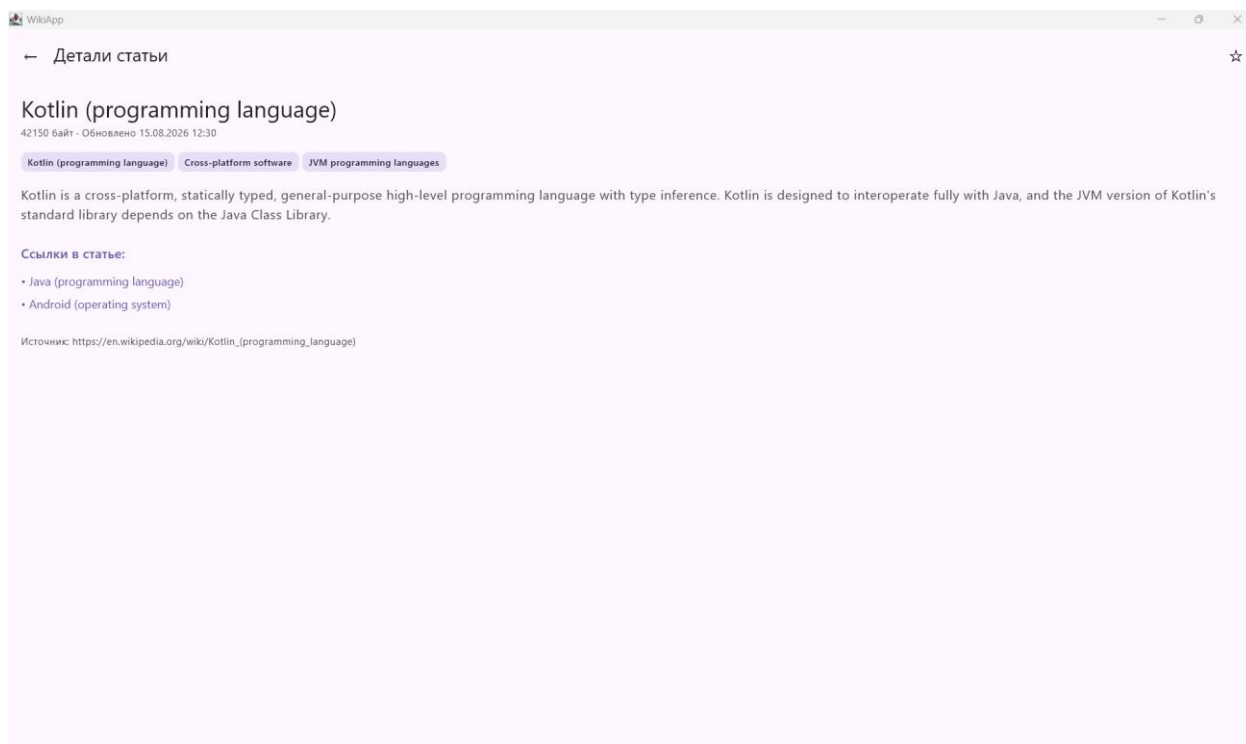


Рисунок 4 – Детали статьи

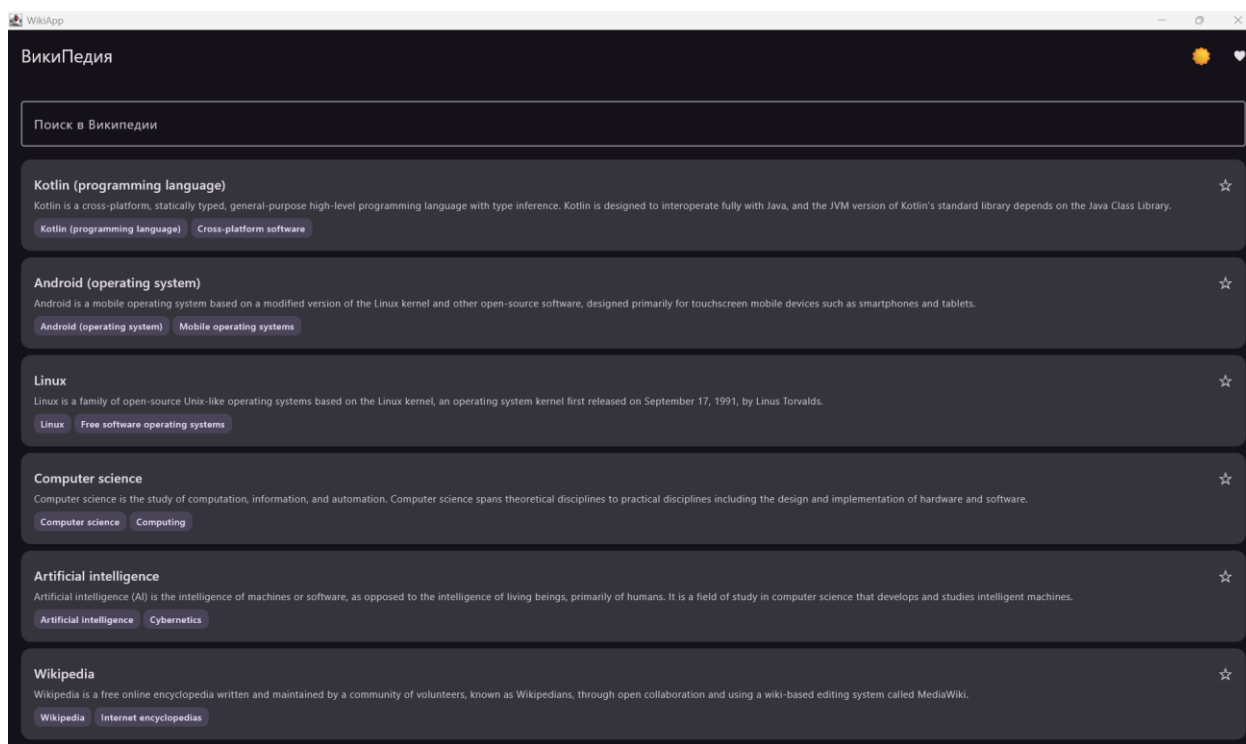


Рисунок 5 – Список статей в тёмном режиме

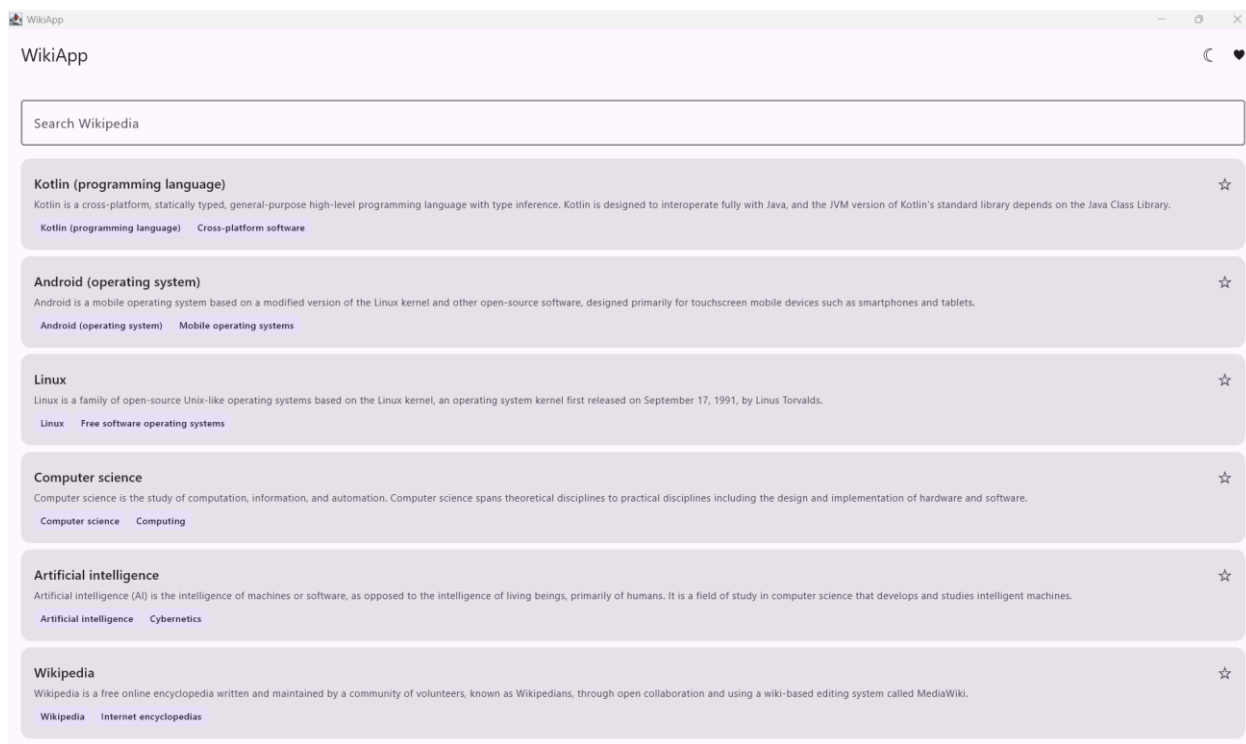


Рисунок 6 – Список статей в английской версии

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы поставленная цель достигнута: разработан каркас кроссплатформенного приложения «ВикиПедия» на Compose Multiplatform. Все поставленные задачи выполнены.

Данные получаются через интерфейс репозитория, поэтому замена моков на реальный источник затронет только его реализацию. Реализованы экран-список с поиском и подгрузкой, экран статьи со связанными статьями и экран избранного; навигация между ними построена на Navigation 3. Экраны получают состояние и обработчик намерений, а не ViewModel целиком, а ViewModel не знает о навигации. Цвета берутся из темы, подписи интерфейса вынесены в строковые ресурсы на английском и русском языках, согласованность локалей проверяется скриптом.

Полученный каркас служит основой для следующей вехи курса, на которой моки будут заменены запросами к реальному источнику данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Compose Multiplatform: официальная документация [Электронный ресурс]. — URL: <https://www.jetbrains.com/help/kotlin-multiplatform-dev/compose-multiplatform.html>
2. Navigation 3: официальная документация [Электронный ресурс]. — URL: <https://developer.android.com/guide/navigation/navigation-3>
3. MediaWiki Action API: документация [Электронный ресурс]. — URL: https://www.mediawiki.org/wiki/API:Main_page
4. Material Design 3: цветовые схемы [Электронный ресурс]. — URL: <https://m3.material.io/styles/color/system>
5. Compose compiler reports [Электронный ресурс]. — URL: <https://developer.android.com/develop/ui/compose/performance/stability/diagnose>